

TRACE: Rollout-Guided Quantization-Aware Training for FP4 Reinforcement Learning of MoE Language Models

Xin Wang^{1,2,*} Hao Yu^{1,*} Zhengyang Zhuge^{1,*} Bochao Mao¹
 Zheng Li¹ Junda Feng¹ Yuyan Luo¹ Yi Zhang¹
 Yizhong Cao^{1,†} Mi Zhang^{2,†} Dayiheng Liu¹ Jianwei Zhang^{1,†}

¹Alibaba Token Hub, Alibaba Group, ²Ohio State University

Abstract

Reinforcement learning (RL) for post-training large language models (LLMs) incurs substantial computation and memory overhead during rollout generation, which motivates low-precision rollout for efficient RL training. However, existing FP4 RL methods suffer from a key limitation: they primarily optimize quantization accuracy on the training and rollout paths independently rather than directly reducing the discrepancy between the two quantized execution paths. In this work, we propose TRACE (Train-Rollout Quantization Alignment via Compact Guidance), an FP4 quantization framework for RL training of Mixture-of-Experts (MoE) language models that addresses the limitation of existing FP4 RL methods. TRACE incorporates rollout-guided quantization-aware training that uses rollout-side quantization outcomes to guide training-side FP4 rounding decisions, directly reducing train-rollout discrepancy. Moreover, TRACE adopts an efficient quantization-information caching scheme that selectively retains mantissa and scale information from deeper layers to reduce the storage and communication overhead introduced by rollout guidance. We evaluate TRACE on four large-scale MoE language models across reasoning, coding, and long-horizon RL tasks. Our results demonstrate that TRACE enables joint FP4 weight/activation and FP4 KV-cache rollout with RL performance comparable to BF16 rollout, while achieving up to $5.4\times$ rollout speedup and strong final FP4 performance compared with post-hoc FP4 quantization of BF16-trained policies.

1 Introduction

Reinforcement learning (RL) has become an important post-training paradigm for improving the reasoning, coding, and other complex capabilities of Large Language Models (LLMs) (Zhao et al., 2023; Zhang et al., 2025). Despite its effectiveness, RL training incurs substantial computation and memory overhead due to the repeated generation of long trajectories during rollout (Wang et al., 2025). Low-precision rollout (Zhuge et al., 2026; Yao et al., 2026; Li et al., 2026; Xi et al., 2026; Huang et al., 2025) provides a promising solution by quantizing the policy model during generation to reduce rollout cost. In particular, aggressive FP4 quantization (W4A4) provides greater opportunities to accelerate rollout generation and reduce its memory footprint. However, the coarse quantization space of FP4 also introduces large numerical discrepancies between the rollout and training execution paths, which can result in policy mismatch and destabilize RL training. Such inconsistency is particularly challenging for Mixture-of-Experts (MoE) language models (Shazeer et al., 2017), where numerical differences may further interact with expert routing and amplify the mismatch between rollout and training policies (Zheng et al., 2025a). To mitigate this issue, several recent FP4 RL methods (Zhuge et al., 2026; Mak et al., 2026) improve the numerical consistency between low-precision rollout and training through quantization-aware training or quantization-error compensation.

Despite these efforts, existing FP4 RL methods suffer from a fundamental issue: **Misalignment between quantization accuracy and train-rollout discrepancy**. Existing methods primarily improve quantization accuracy with respect to the high-precision representation on the training and rollout paths independently. However, minimizing the quantization error on each execution path does not directly minimize the discrepancy between the two quantized execution paths. As a consequence, even when both paths achieve small quantization errors, their quantized values can still diverge substantially, leading to large policy mismatch, degraded RL performance, or even training collapse.

*Equal contribution.

†Corresponding author.

‡Work is done during internship at Alibaba Token Hub, Alibaba Group.

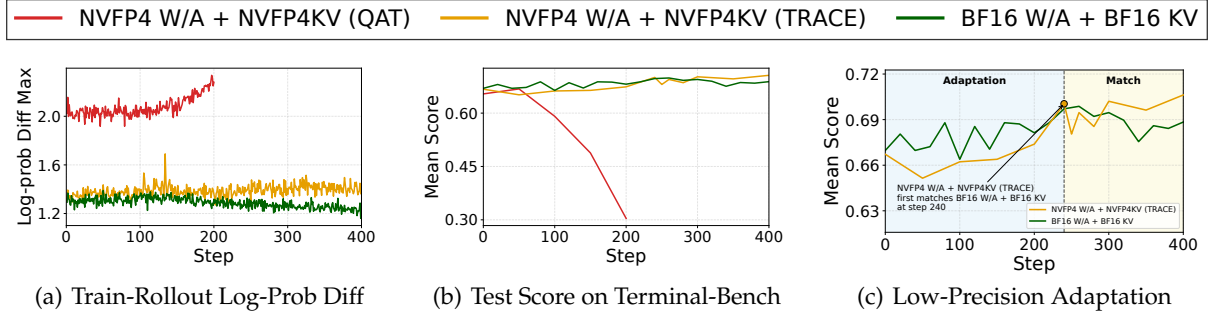


Figure 1: Training dynamics of Qwen3.8-Flash-Next under FP4 rollout. Conventional FP4 QAT exhibits increasing train-rollback discrepancy and degraded test score on Terminal-Bench 2.1, while TRACE maintains stable alignment and enables the policy to progressively adapt to FP4 rollout.

In this paper, we propose TRACE (Train-Rollout Quantization Alignment via Compact Guidance), an FP4 quantization framework for RL training of MoE language models. TRACE differs from existing FP4 RL approaches in two key aspects. **① Rollout-Guided Quantization-Aware Training:** TRACE directly reduces local quantized train-rollback discrepancy by using rollout-side quantization outcomes to guide training-side FP4 rounding decisions. **② Efficient Quantization-Information Caching:** To reduce the storage and communication overhead introduced by rollout guidance during RL training, TRACE only retains the mantissa and scale information from selected deeper layers.

Figure 1 highlights the training dynamics under aggressive FP4 rollout (NVFP4 W/A + NVFP4 KV) with TRACE. FP4 QAT exhibits an increasingly large train-rollback policy mismatch as training proceeds, resulting in a sharp degradation in test score on Terminal-Bench. In contrast, TRACE effectively controls the train-rollback discrepancy and maintains stable RL training. More interestingly, once the discrepancy is properly controlled, the policy progressively adapts to FP4 rollout: although TRACE initially suffers from quantization-induced performance degradation, the performance gap relative to BF16 rollout gradually diminishes during training and can eventually be closed or even reversed.

We compare TRACE with representative FP4 rollout baselines, including vanilla QAT, QaRL, QUADS and Score Centering, as well as post-hoc FP4 quantization methods, including vanilla NVFP4, 4over6, and H-Scale. To demonstrate the generalizability of TRACE, we evaluate it on four MoE language models—Qwen3.5-35B-A3B, Qwen3.5-122B-A10B, Qwen3.8-Flash-Next, and Qwen3.8-2.4T-A95B—across reasoning, coding, and long-horizon RL tasks. We highlight three of our findings:

- TRACE enables joint FP4 W/A and FP4 KV rollout while preserving BF16-level performance, maintaining training dynamics and final task performance comparable to BF16 rollout.
- TRACE retains the efficiency advantage of FP4 rollout with limited algorithm overhead, achieving up to $5.4\times$ rollout speedup over BF16 rollout.
- TRACE achieves stronger final FP4 task performance than pipelines that first train with BF16 rollout and apply post-hoc FP4 quantization.

2 Related Work

Reinforcement Learning for MoE Language Models: Modern LLM post-training increasingly relies on reinforcement learning (RL), with GRPO (Shao et al., 2024) being widely adopted for reasoning models. For Mixture-of-Experts (MoE) models (Shazeer et al., 2017), RL training is particularly sensitive to train-rollback inconsistency because small numerical differences can alter router scores and activate different experts (Zheng et al., 2025a). R3 (Ma et al., 2025) mitigates this routing mismatch by replaying rollout routing decisions during training, while PR2 (Dong et al., 2026) further accounts for router evolution across policy updates. GSPO (Zheng et al., 2025b) instead improves stability through sequence-level importance ratios and clipping. Meanwhile, the large parameter capacity of MoE models introduces substantial memory, communication, and rollout costs, motivating efficient distributed training and low-precision rollout (Wang et al., 2025; 2024).

Low-Precision Reinforcement Learning: Recent works have explored low-precision rollout to reduce the generation cost of RL for LLMs. For MoE models, quantization is often applied only to routed experts while keeping other modules in higher precision (Zhuge et al., 2026; Yao et al., 2026; Yu et al., 2026). Most prior methods focus on 8-bit precision. For example, QuRL (Li et al., 2026) reduces train-rollback mismatch through adaptive clipping and update-aware quantization, while Jet-RL (Xi et al., 2026) adopts a unified FP8 computation flow. More recent works extend rollout quantization to 4-bit precision. QeRL (Huang

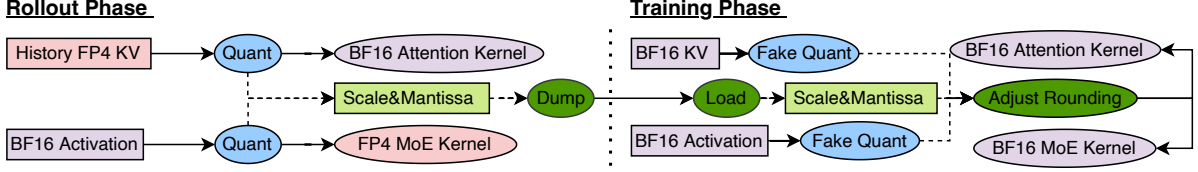


Figure 2: Overview of TRACE for FP4 RL (FP4 W/A + FP4 KV) Training of MoE Models .

et al., 2025) mainly considers 4-bit weight quantization with higher-precision activations; QUADS (Zhuge et al., 2026) combines asymmetric QAT with rollout-side activation compensation; Rollout-ResQ (Mak et al., 2026) corrects FP4 rollout errors with sparse residuals; and QaRL (Gu et al., 2026) aligns training and rollout through matched low-precision kernels. Orthogonally, Score Centering (Marek and Ryabinin, 2026) stabilizes RL by correcting optimization drift induced by training–inference mismatch, without directly aligning the underlying low-precision computations. However, even with matched execution or optimization-level correction, small activation differences can still cross FP4 rounding boundaries and produce different codewords. In contrast, TRACE directly uses rollout-side FP4 outcomes to guide training-side rounding, explicitly targeting this rounding–outcome mismatch.

3 Train-Rollout Quantization Alignment via Compact Guidance (TRACE)

Figure 2 provides an overview of TRACE. At a high level, TRACE is an FP4 quantization framework for efficient RL training of MoE language models that directly aligns the quantized computation paths used for rollout generation and training. Specifically, during rollout generation, TRACE records the quantization outcomes of FP4 routed experts’s activation and FP4 KV states. During the subsequent quantization-aware training (QAT) phase, the recorded rollout-side information is used to guide the corresponding training-side rounding decisions. To reduce the overhead of rollout guidance, TRACE further adopts an efficient quantization-information caching scheme. It retains only the mantissa and scale information from selected deeper layers to reduce storage and communication costs. In the following, we describe rollout-guided quantization-aware training and efficient quantization-information caching in detail. The pseudocode of TRACE is provided in Section A.1.

3.1 Rollout-Guided Quantization-Aware Training

Motivation: Low-precision rollout introduces numerical discrepancy between the training and rollout execution paths. Simply combining FP4 quantization-aware training (QAT), where FP4 fake quantization is applied during the forward pass while the backward pass remains in BF16, with FP4 rollout generation can therefore lead to substantial policy divergence and degraded or even unstable RL training (Zhuge et al., 2026). For paired training- and rollout-side activations corresponding to the same trajectory, token position, and quantization site, we characterize their quantized discrepancy as

$$\mathcal{D}_{\text{act}} = \left\| Q_{\text{FP4}}^{\text{train}}(X_{\text{train}}) - Q_{\text{FP4}}^{\text{rollout}}(X_{\text{rollout}}) \right\|_F, \quad (1)$$

where X_{train} and X_{rollout} denote the corresponding activations produced during training and rollout generation, respectively, and $Q_{\text{FP4}}^{\text{train}}$ and $Q_{\text{FP4}}^{\text{rollout}}$ denote the FP4 quantization procedures on the two execution paths. We use $\mathcal{D}_{\text{act}}$ to characterize local train–rollout discrepancy.

Existing methods such as QUADS (Zhuge et al., 2026) and Rollout-ResQ (Mak et al., 2026) mitigate this discrepancy indirectly by improving the quantization accuracy of each execution path. Specifically, they primarily reduce the quantization error with respect to the corresponding high-precision representation, e.g., $\mathcal{L}_{\text{error}}(X) = \|X - Q(X)\|_F$. However, minimizing the quantization

	2	X_{train}	X_{rollout}	3	Vanilla FP4	QUADS	TRACE
①		2.40	2.49		2 2 ✓	2.4 2.5 ✗	2 2 ✓
		Discrepancy: 0.09			Discrepancy: 0	Discrepancy: 0.1	Discrepancy: 0
②		2.01	2.99		2 3 ✗	2.01 3.0 ✗	3 3 ✓
		Discrepancy: 0.98			Discrepancy: 1	Discrepancy: 0.99	Discrepancy: 0

Figure 3: Two Examples of Train–Rollout Discrepancy under Different FP4 RL Methods.

errors of the training and rollout paths independently does not directly minimize their cross-path discrepancy. To illustrate this issue, we provide two examples of QUADS in Figure 3. QUADS retains the BF16 training activation and compensates for the FP4 rollout quantization error using a residual $R(X) = X - Q(X)$. In example ①, the training and rollout activations are 2.40 and 2.49, respectively. Vanilla FP4 maps both values to 2, yielding zero quantized discrepancy. In contrast, QUADS retains 2.40 on the training side and reconstructs the rollout activation as 2.50, reducing the per-path quantization

error but increasing the train–rollout discrepancy to 0.10. In example ②, QUADS similarly reduces the rollout-side quantization error but leaves a discrepancy of 0.99, close to the discrepancy of 1.0 under vanilla FP4. These examples demonstrate that improving per-path quantization accuracy does not necessarily reduce train–rollout discrepancy.

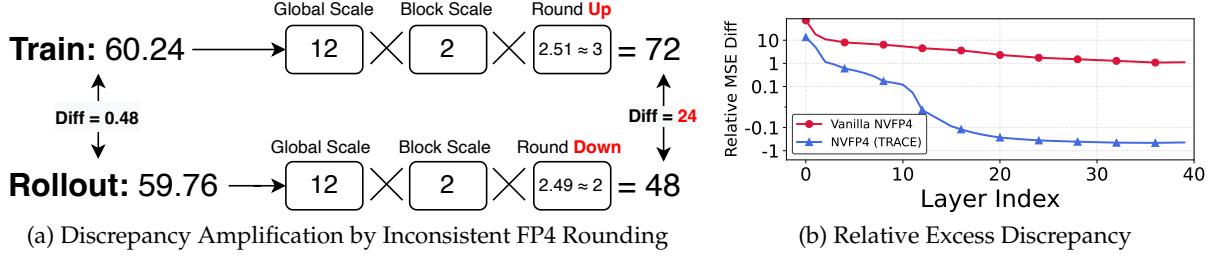


Figure 4: Problem Analysis of FP4 RL Methods on Qwen3.5-35B-A3B.

Key Design: The key idea of TRACE is to reduce the additional train–rollout discrepancy introduced by inconsistent FP4 rounding. A key source of this discrepancy is that small differences between the BF16 training and rollout activations can be substantially amplified when they are rounded to different FP4 codewords. As illustrated in Figure 4(a), the original training and rollout activations, 60.24 and 59.76, differ by only 0.48. After applying the same global and block scales, however, their normalized values, 2.51 and 2.49, fall on opposite sides of the rounding boundary and are rounded to 3 and 2, respectively. Consequently, the discrepancy is amplified from 0.48 in BF16 to 24 after FP4 quantization. We further quantify this quantization-induced discrepancy in Figure 4(b) using the relative excess train–rollout MSE, $(D_{\text{FP4}} - D_{\text{BF16}})/D_{\text{BF16}}$, where D_{FP4} and D_{BF16} denote the train–rollout activation MSE under FP4 and BF16 computation, respectively. As shown, vanilla NVFP4 introduces substantial additional discrepancy across model layers.

Based on this, TRACE uses the rollout-side quantization outcome to guide training-side rounding. For each captured training-side activation, let $\{q_-, q_+\}$ denote its two neighboring normalized FP4 E2M1 codewords under the rollout-side scale, and let q_{rollout} denote the exact FP4 codeword produced by the corresponding rollout-side activation. Instead of applying standard round-to-nearest (RTN), TRACE selects

$$q_{\text{TRACE}} = \arg \min_{q \in \{q_-, q_+\}} |q - q_{\text{rollout}}|. \quad (2)$$

Let $q_{\text{RTN}} \in \{q_-, q_+\}$ denote the codeword selected by standard RTN under the same scale and candidate set. By construction,

$$|q_{\text{TRACE}} - q_{\text{rollout}}| \leq |q_{\text{RTN}} - q_{\text{rollout}}|. \quad (3)$$

Therefore, under the full-information formulation, relative to RTN, TRACE does not increase the local quantized train–rollout discrepancy at each captured site under the same candidate set.

This local guarantee does not imply monotonic reduction of the full-network discrepancy in Equation (1) or of policy-level divergence after downstream nonlinear transformations. Rather, TRACE reduces unnecessary amplification caused by inconsistent FP4 rounding. As illustrated in Figure 3, rollout-guided rounding eliminates the quantized discrepancy in both examples, while Figure 4(b) shows substantially reduced train–rollout discrepancy across model layers. The formulation above assumes access to the exact rollout-side FP4 codeword. In Section 3.2, we introduce a compressed caching scheme that approximates this guidance using only partial rollout-side code information.

3.2 Mantissa-Only Train–Rollout Communication

Motivation: Although rollout-guided QAT effectively reduces train–rollout discrepancy, preserving complete rollout-side quantization information introduces substantial data-movement overhead. As illustrated in Figure 5, the activation-side and KV-side quantization records follow different collection paths during rollout. For activation guidance, the intermediate quantization information generated during each forward is first written into a temporary GPU buffer and asynchronously offloaded to a CPU-side cache after the forward completes. The temporary GPU buffer is reused by subsequent forwards. In contrast, the quantized KV states are already retained in the persistent FP4 KV cache and therefore only need to be gathered and transferred to CPU memory when the corresponding request finishes. After request completion, both activation and KV quantization records are serialized and persisted to storage. Before QAT, the training engine reloads the records for the forward microbatches into CPU memory, after which the corresponding slices are transferred to GPUs.

The amount of data generated by this full-information pipeline can be extremely large. For example, consider Qwen3.5-35B-A3B with hidden size $D_1 = 2048$, KV head dimension $D_2 = 256$, two KV heads,

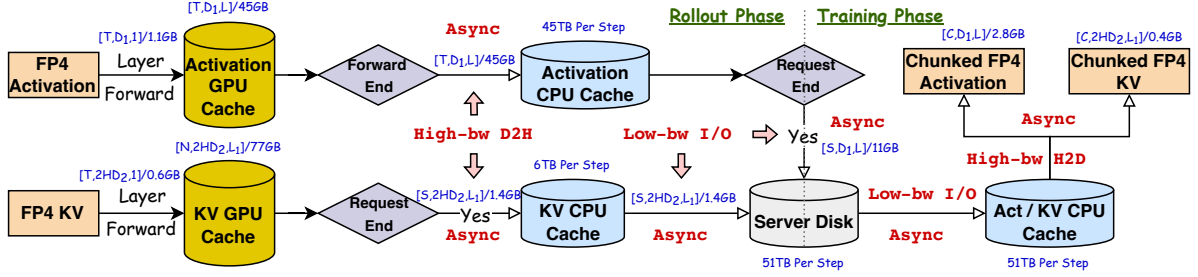


Figure 5: Data Communication Pipeline of Rollout-Side Quantization Information.

40 MoE layers, 10 full-attention layers, and a maximum response length of 256K tokens. Assuming 4.5 bits are retained per quantized value, one generated token produces approximately 45 KB of activation guidance and 5.6 KB of KV guidance, or about 50 KB in total. With 4,096 trajectories in one RL step, this corresponds to up to approximately 51 TB of rollout-side quantization information per step. Such a data volume is difficult to hide through asynchronous communication alone. For illustration, assuming an effective GPU-to-CPU bandwidth of 300 GB/s, transferring 51 TB requires approximately three minutes in aggregate. In contrast, at a representative storage bandwidth of 5 GB/s, writing or reading the same amount of data would require roughly three hours if performed serially, far exceeding the wall-clock time of a typical RL step. Therefore, even if GPU-to-CPU transfer can be efficiently overlapped with rollout computation, storage I/O can easily become the dominant bottleneck once the rollout-side quantization records are persisted between rollout and training.

Key Design: The key idea of TRACE is to communicate only the rollout-side quantization information necessary to determine the desired training-side rounding direction. As shown in Figure 6, for over 99% of the mismatched quantized values across layers, the training and rollout results differ by only one adjacent FP4 codebook entry. Because the two quantized values are overwhelmingly adjacent in the FP4 codebook, the training-side activation together with the rollout scale strongly constrains the candidate codewords. Consequently, a small amount of mantissa code information is typically sufficient to identify the desired neighboring rollout-side codeword. Therefore, communicating the complete quantized value is unnecessary. We further observe that rounding corrections toward lower FP4 codewords occur predominantly in deeper layers, indicating that rollout-side quantization information from these layers is more important for train-rollout alignment. Based on these observations, TRACE communicates only the mantissa and scale information from the latter half of the model layers. During rollout generation, this compact information is cached and transferred to the training engine, where it is used to reconstruct a compact rollout-side reference and guide training-side quantization. This mantissa-only train-rollout communication substantially reduces the storage and communication overhead while preserving the effectiveness of TRACE.

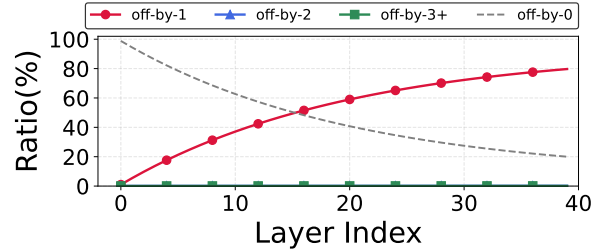


Figure 6: Difference in FP4 Codebook Entries Between Training and Rollout Activations.

4 Experiment Analysis

Baselines. We compare TRACE against three groups of methods: (1) NVFP4 rollout methods, including vanilla NVFP4, NVFP4 with quantization-aware training (QAT), QaRL (Gu et al., 2026), QUADS (Zhuge et al., 2026) (Section 4.1), and Score Centering (Marek and Ryabinin, 2026) (Section 4.4); (2) MXFP4 rollout methods, including vanilla and QAT variants under W4A8 and W4A4 settings (Section 4.3); and (3) NVFP4 post-training quantization methods, including vanilla NVFP4, 4over6 (Cook et al., 2025), and H-Scale (Yu et al., 2026) (Section 4.5).

Models and Datasets. We evaluate TRACE on four MoE language models: Qwen3.5-35B-A3B and Qwen3.5-122B-A10B (Qwen Team, 2026a), Qwen3.8-Flash-Next (Qiu et al., 2026) (125B total / 6B activated parameters), and Qwen3.8-2.4T-A95B (Qwen Team, 2026b). We consider three RL settings: reasoning, coding, and long-horizon tasks. For reasoning RL, we train Qwen3.5-35B-A3B and evaluate it on LiveCodeBench v6 (Jain et al., 2025), AIME 2024/2025 (Mathematical Association of America, 2024; 2025), and HMMT 2025 (Balunović et al., 2025). For coding RL, we train Qwen3.5-122B-A10B and Qwen3.8-Flash-Next, and evaluate them on DeepSWE v1.1 (Huang et al., 2026) and Terminal-Bench 2.1 (Merrill et al., 2026), respectively. For long-horizon RL, we train Qwen3.8-2.4T-A95B and evaluate it on GDPval (Patwardhan et al., 2025). We report sampled average pass@1 over 8 responses for AIME24, AIME25, and

Table 1: Performance ($\uparrow$) comparison of QAT, QaRL, QUADS, and TRACE under joint NVFP4 W/A/KV rollout setting on Qwen3.5-35B-A3B for reasoning RL tasks. The best performance is marked in bold. The performance gain compared to baselines is marked in green inside bracket.

ROLLOUT CONFIG	ALGO.	QWEN3.5-35B-A3B				
		LIVECODEBENCH	AIME24	AIME25	HMMT25	AVERAGE
BF16 W/A + BF16 KV	Vanilla	67.1	83.8	81.3	67.5	74.9
NVFP4 W/A + NVFP4 KV	QAT	55.1	70.7	63.4	49.2	59.6
	QaRL	54.1	73.3	57.9	49.2	58.6
	QUADS	60.4	80.5	75.4	59.0	68.8
	TRACE	66.4 ($\uparrow 6.0$)	86.3 ($\uparrow 5.8$)	78.5 ($\uparrow 3.1$)	70.0 ($\uparrow 11.0$)	75.3 ($\uparrow 6.5$)

HMMT25, and over 10 responses for LiveCodeBench. For Terminal-Bench, DeepSWE, and GDPval, we report scores averaged over 10, 4, and 3 independent runs per task, respectively.

Implementation Details. For fair comparison, all methods use the same versions of VeRL (Sheng et al., 2025), Megatron (Shoeybi et al., 2019), and SGLang (Zheng et al., 2024), where VeRL coordinates asynchronous RL training, Megatron serves as the training backend, and SGLang performs rollout generation. We use disagg architecture to deploy the trainer and rollout engines. We adopt GRPO (Shao et al., 2024) for policy optimization and apply R3 (Ma et al., 2025) to replay rollout-side expert-routing decisions during training. We additionally use token-level importance ratios between the training and rollout policies (Zheng et al., 2025a), with the clipping upper bound set to 5, and set max_version_diff=3 to bound policy staleness introduced by asynchronous training. For rollout-guided quantization, rollout-side quantization records are paired with the corresponding training-side activations by trajectory, token position, and quantization site. Since the training policy can be newer than the policy that generated the rollout, TRACE does not eliminate the underlying activation difference caused by policy staleness; instead, it reduces the additional discrepancy introduced when such differences are amplified by inconsistent FP4 rounding. Unless otherwise specified, we quantize only the routed experts in MoE layers and the KV cache in softmax-attention layers during rollout, while keeping all other modules in BF16. For Qwen3.5-35B-A3B, Qwen3.5-122B-A10B, Qwen3.8-Flash-Next, and Qwen3.8-2.4T-A95B, each RL step samples 256×16 , 128×16 , 64×16 , and 64×8 trajectories, respectively. We set the maximum response length to 32K tokens for reasoning RL and the maximum context length to 256K tokens for coding and long-horizon RL. Qwen3.5-35B-A3B and Qwen3.8-Flash-Next are trained for 400 RL steps, while Qwen3.5-122B-A10B and Qwen3.8-2.4T-A95B are trained for 50 steps.

4.1 Overall Comparison

We first evaluate the performance of TRACE under joint FP4 weight and KV cache quantization during rollout from three aspects: (1) performance across different benchmarks on the reasoning RL task, (2) performance on larger-scale MoE models under coding and long-horizon RL tasks, and (3) performance evolution throughout RL training. We compare TRACE with baselines under the same FP4 rollout configuration and use BF16 rollout as the high-precision reference.

Performance on Different Benchmarks. We evaluate TRACE on Qwen3.5-35B-A3B across reasoning benchmarks. As shown in Table 1, TRACE consistently outperforms QAT, QaRL, and QUADS across all four benchmarks. In particular, TRACE improves the average score from 68.8 to 75.3. The improvement is especially significant on HMMT25, where the score increases from 59.0 to 70.0. More importantly, TRACE achieves an average score matching the BF16 rollout, demonstrating that TRACE can effectively recover the performance degradation introduced by joint FP4 weight and KV cache rollout while achieving performance comparable to BF16 rollout.

Table 2: Performance ($\uparrow$) on larger-scale MoE LLMs, including Qwen3.5-122B-A10B and Qwen3.8-Flash-Next (125B-A6B) trained on coding RL tasks and evaluated on DeepSWE and Terminal-Bench, respectively. Qwen3.8-2.4T-A95B is trained on long-horizon RL task and evaluated on GDPval. The best score in each row is marked in bold. The performance gain compared to baselines is marked in green inside bracket.

MODEL	BF16	QAT	QUADS	TRACE
QWEN3.5-122B-A10B	33.4	28.8	29.1	33.0 ($\uparrow 3.9$)
QWEN3.8-FLASH-NEXT	68.8	60.4	66.4	70.6 ($\uparrow 4.2$)
QWEN3.8-2.4T-A95B	90.3	88.7	85.9	90.2 ($\uparrow 1.5$)

Performance on Larger-Scale Models. To examine the generalizability of TRACE to larger-scale MoE models, we further evaluate it on Qwen3.5-122B-A10B, Qwen3.8-Flash-Next, and Qwen3.8-2.4T-A95B under coding and long-horizon RL settings. As shown in Table 2, TRACE consistently outperforms QAT across all three models. In particular, on Terminal-Bench trained with Qwen3.8-Flash-Next, TRACE improves the score from 66.4 to 70.6. Moreover, TRACE matches BF16 rollout performance across all three

models, achieving 33.0 vs. 33.4 on DeepSWE, 70.6 vs. 68.8 on Terminal-Bench, and 90.2 vs. 90.3 on GDPval. These results demonstrate that the effectiveness of TRACE generalizes across different model scales and RL tasks while preserving the performance of high-precision BF16 rollout.

Training Dynamics Analysis. We examine the training dynamics of Qwen3.5-35B-A3B throughout RL training on reasoning tasks. The analysis on Qwen3.8-Flash-Next and Qwen3.8-2.4T-A95B are provided in Section A.2 and Section A.3 respectively. As shown in Figure 8, we make two observations.

(1) QAT, QaRL and QUADS exhibit increasingly large train-rollout discrepancy as training proceeds, which is eventually accompanied by clear signs of training instability. In particular, the mean reward starts to decrease substantially at later training stages, while the test scores on all reasoning benchmarks degrade sharply. In contrast, TRACE effectively controls the train-rollout discrepancy and maintains training dynamics close to BF16 rollout. Both the training reward and downstream test performance remain stable and continue to improve throughout training. These results further indicate that reducing train-rollout discrepancy is important for maintaining stable RL optimization under aggressive FP4 rollout. (2) More importantly, we observe an interesting adaptation behavior under FP4 rollout with TRACE. As illustrated in Figure 7, the test score of TRACE is initially lower than that of the BF16 rollout due to the accuracy degradation introduced by quantization. However, **this performance gap gradually diminishes as RL training proceeds**. After approximately 120 training steps, TRACE catches up with the BF16 reference and subsequently remains comparable, with higher point estimates at several later steps. This trend suggests that TRACE does not merely preserve the initial model quality under FP4 execution. Instead, once the train-rollout discrepancy is sufficiently controlled, the policy can progressively adapt to the low-precision execution environment during RL training, recovering the initial quantization-induced degradation and eventually matching the BF16 rollout.

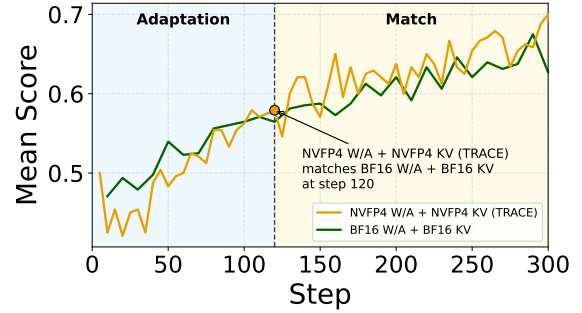


Figure 7: Adaptation for NVFP4 (W/A+KV) with TRACE on HMMT25 during RL training.

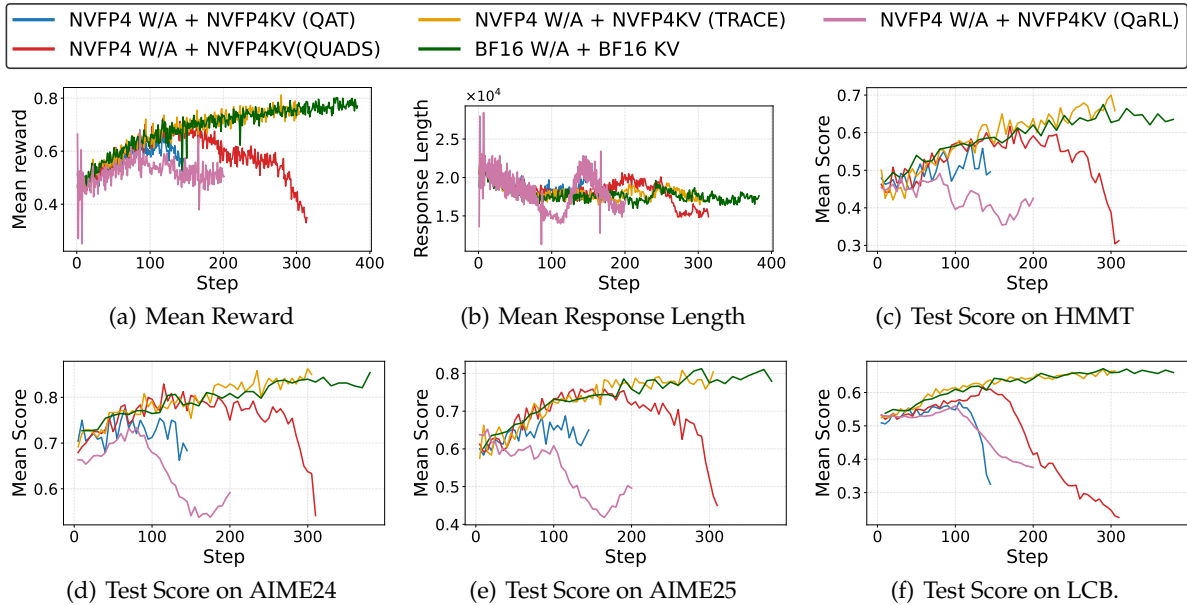


Figure 8: Training dynamics of Qwen3.5-35B-A3B on reasoning RL tasks, including reward, response length, and test scores on HMMT, AIME24, AIME25, and LiveCodeBench.

4.2 Efficiency Analysis

We next evaluate the efficiency of TRACE on Qwen3.5-35B-A3B from two aspects: (1) rollout decoding throughput under different generation lengths, and (2) end-to-end RL training efficiency. We also analyze the efficiency of Qwen3.8-Flash-Next in Section A.4.

Rollout Efficiency. We first evaluate the decoding throughput of TRACE under different output lengths. As shown in Figure 9(a), the full rollout-guided variant incurs substantial throughput degradation due to the overhead of storing and transferring complete rollout-side quantization information. In contrast, TRACE

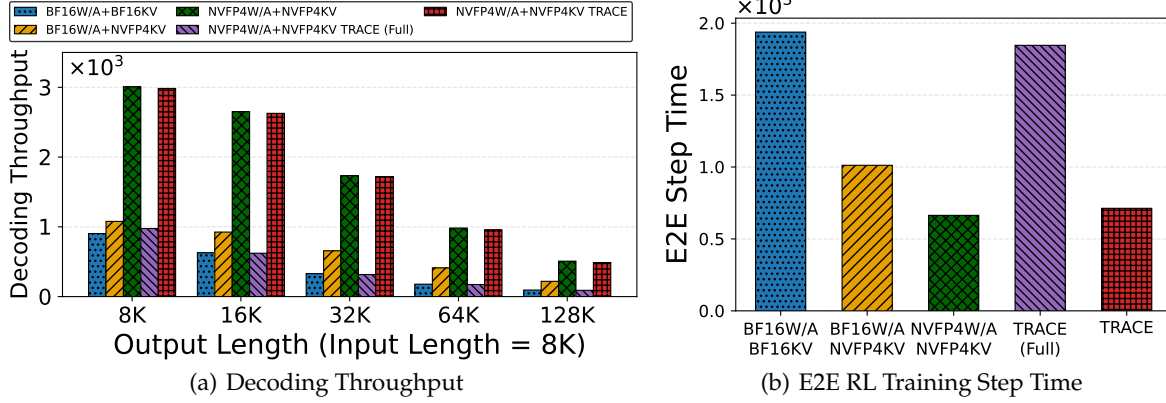


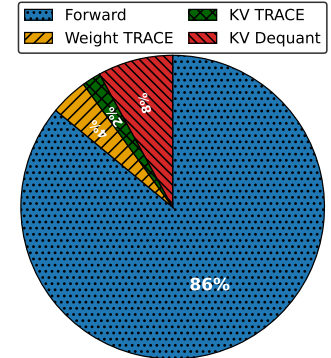
Figure 9: Efficiency analysis of TRACE on Qwen3.5-35B-A3B. Decoding throughput is measured using only the rollout engine (SGLang) on 4 GB200 GPUs. End-to-end RL training step time is measured on the reasoning RL tasks using 72 GB200 GPUs.

Table 3: Ablation of isolated NVFP4 weight and NVFP4 KV on Qwen3.5-35B-A3B under reasoning RL tasks. † marks results quoted from QUADS (Zhuge et al., 2026). The best performance is marked in bold. The performance gain compared to baselines is marked in green inside bracket.

ROLLOUT CONFIG	ALGO.	LIVECODEBENCH	AIME24	AIME25	HMMT25	AVERAGE
BF16 W/A + BF16 KV	Vanilla	67.1	83.8	81.3	67.5	74.9
NVFP4 W/A + BF16 KV	QAT†	63.5	81.7	78.8	60.4	71.1
	QUADS†	64.8	83.3	80.4	62.9	72.9
	TRACE	67.1 (↑2.3)	83.1	82.1 (↑1.7)	69.4 (↑6.5)	75.4 (↑2.5)
BF16 W/A + NVFP4 KV	QAT	65.3	82.3	79.2	65.8	73.2
	TRACE	68.1 (↑2.8)	83.0 (↑0.7)	80.7 (↑1.5)	67.3 (↑1.5)	74.8 (↑1.6)

maintains throughput close to vanilla FP4 rollout across different output lengths. For Qwen3.5-35B-A3B, retaining one-bit mantissa information together with amortized scale metadata for the latter 20 layers consumes approximately $2048 \times 20 \times 1.5 = 61,440$ bits, or 7.5 KB, of cached quantization information per generated token, which reduces the cache size by approximately $6\times$ compared with caching the full quantized activation. In particular, TRACE achieves up to $5.4\times$ higher decoding throughput at 128K output length compared with BF16 rollout, demonstrating that the proposed quantization-information caching preserves most of the decoding efficiency of FP4 rollout while enabling rollout-guided training.

End-to-End RL Training Efficiency. Lastly, we evaluate the end-to-end training overhead introduced by TRACE. As shown in Figure 9(b), TRACE increases the RL step time from 664 to 713, corresponding to only 7.4% overhead compared with vanilla joint FP4 rollout. To further understand the source of this overhead, Figure 10 reports the rollout-time breakdown of TRACE. The model forward pass accounts for 86% of the rollout time, while weight-side and KV-side rollout guidance account for only 4% and 2%, respectively; the remaining 8% comes from KV dequantization. These results demonstrate that TRACE introduces limited runtime overhead while retaining the efficiency advantage of FP4 rollout.



4.3 Ablation Study

Isolated Weight and KV Quantization. We first examine the individual impact of FP4 weight and KV cache quantization. As shown in Table 3, TRACE outperforms the baselines on most benchmarks in both isolated settings and consistently achieves performance comparable to BF16 rollout. These results suggest that the joint FP4 configuration is substantially more challenging than either isolated setting, highlighting the importance of controlling the train-rollout discrepancy under joint FP4 rollout.

Performance Under Microscaling FP4 Formats. With recent adoption of MXFP4 MegaMoE kernels for efficient MoE inference, we further examine whether TRACE generalizes to microscaling FP4 formats. As

Figure 10: Rollout time breakdown for TRACE.

Table 4: Performance ($\uparrow$) comparison of stabilizing algorithms with different MXFP4 rollout configurations on Qwen3.5-35B-A3B under the reasoning RL task. The best performance is marked in bold. The performance gain compared to the QAT baseline is marked in green inside brackets.

ROLLOUT CONFIG	ALGO.	LIVECODEBENCH	AIME24	AIME25	HMMT25	AVERAGE
BF16 W/A + BF16 KV	Vanilla	67.1	83.8	81.3	67.5	74.9
W4A8 + MXFP4 KV	QAT	64.9	75.4	76.9	58.8	69.0
	TRACE	67.1 ($\uparrow 2.2$)	83.3 ($\uparrow 7.9$)	81.2 ($\uparrow 4.3$)	68.7 ($\uparrow 9.9$)	75.1 ($\uparrow 6.1$)
W4A4 + MXFP4 KV	QAT	60.2	74.8	73.2	60.7	67.2
	TRACE	65.2 ($\uparrow 5.0$)	82.0 ($\uparrow 7.2$)	79.8 ($\uparrow 6.6$)	67.1 ($\uparrow 6.4$)	73.5 ($\uparrow 6.3$)

Table 5: Modular sensitivity study of TRACE with FP4 MoE and FP4 KV on Qwen3.5-35B-A3B under reasoning RL tasks. TRACE (R-1bit-L20) is the default config. The best performance is marked in bold. The performance gain compared to the QUADS baseline is marked in green inside brackets.

ROLLOUT CONFIG	ALGO.	LIVECODEBENCH	AIME24	AIME25	HMMT25	AVERAGE
BF16 W/A + BF16 KV	Vanilla	67.1	83.8	81.3	67.5	74.9
FP4 W/A + FP4 KV	QUADS	60.4	80.5	75.4	59.0	68.8
	TRACE (R-4bit-L40)	67.0 ($\uparrow 6.6$)	86.1 ($\uparrow 5.6$)	80.1 ($\uparrow 4.7$)	70.5 ($\uparrow 11.5$)	75.9 ($\uparrow 7.1$)
	TRACE (R-3bit-L40)	66.8 ($\uparrow 6.4$)	85.9 ($\uparrow 5.4$)	81.2 ($\uparrow 5.8$)	66.8 ($\uparrow 7.8$)	75.2 ($\uparrow 6.4$)
	TRACE (R-2bit-L40)	67.1 ($\uparrow 6.7$)	86.1 ($\uparrow 5.6$)	80.0 ($\uparrow 4.6$)	67.2 ($\uparrow 8.2$)	75.1 ($\uparrow 6.3$)
	TRACE (R-1bit-L40)	66.4 ($\uparrow 6.0$)	86.3 ($\uparrow 5.8$)	80.5 ($\uparrow 5.1$)	70.0 ($\uparrow 11.0$)	75.8 ($\uparrow 7.0$)
	TRACE (R-1bit-L20)	66.4 ($\uparrow 6.0$)	86.3 ($\uparrow 5.8$)	78.5 ($\uparrow 3.1$)	70.0 ($\uparrow 11.0$)	75.3 ($\uparrow 6.5$)
	TRACE (R-1bit-L10)	66.1 ($\uparrow 5.7$)	82.7 ($\uparrow 2.2$)	81.3 ($\uparrow 5.9$)	66.4 ($\uparrow 7.4$)	74.1 ($\uparrow 5.3$)
	TRACE (R-1bit-L5)	65.8 ($\uparrow 5.4$)	81.2 ($\uparrow 0.7$)	79.7 ($\uparrow 4.3$)	66.9 ($\uparrow 7.9$)	73.4 ($\uparrow 4.6$)

shown in Table 4, TRACE improves the average score from 69.0 to 75.1 under W4A8 MXFP4 with FP4 KV cache, and from 67.2 to 73.5 under the more aggressive W4A4 MXFP4 setting. These results demonstrate that TRACE remains effective across different FP4 formats.

Modular Sensitivity Study. Lastly, we evaluate the sensitivity of TRACE to the amount of rollout-side quantization information retained for training. As shown in Table 5, retaining only one mantissa bit across all 40 layers achieves an average score of 75.8. Keeping this information for only the latter 20 layers incurs small degradation, while reducing the coverage to 10 and 5 layers lowers the average score to 74.1 and 73.4, respectively. These results show that TRACE preserves strong performance with only compact mantissa information from a subset of deeper layers.

4.4 Comparison with Score Centering Algorithm

We compare TRACE with Score Centering (SC) (Marek and Ryabinin, 2026) under the same joint NVFP4 W/A/KV rollout configuration. We replace the default policy loss with TIS+SC, combining SC with token-level truncated importance sampling (TIS) weights capped at 2. The centering correction is approximated using rollout policy’s top-128 tokens, their probabilities under both the training and rollout policies, and the remaining probability mass.

As shown in Figure 11, Score Centering indeed prevents the severe training collapse observed with vanilla QAT as illustrated in Section 4.1. However, it leaves a substantially larger train–rollout log-probability discrepancy throughout training: the maximum difference remains around 1.8–2.1, while the minimum stays near -4 ,

Table 6: Performance ($\uparrow$) of SC and TRACE on Qwen3.5-35B-A3B under joint NVFP4 W/A/KV rollout. SC is selected by the highest average across the four benchmarks through 400 steps. The best score in each column is marked in bold.

METHOD	LIVECODEBENCH	AIME24	AIME25	HMMT25	AVERAGE
BF16	67.1	83.8	81.3	67.5	74.9
TIS+SC	66.7	81.7	81.3	66.7	74.1
TRACE	66.4	86.3 ($\uparrow 4.6$)	78.5	70.0 ($\uparrow 3.3$)	75.3 ($\uparrow 1.2$)

both far from the BF16 reference. In contrast, TRACE keeps the train–rollout log-probability difference close to the BF16 rollout throughout training. We additionally compare against SC’s best evaluated checkpoint, selected by the highest average score across the four benchmarks. As shown in Table 6, TRACE achieves an average score of 75.3 versus 74.1 for SC at step 380, a gain of 1.2 percentage points. These results suggest that by directly reducing the quantization-induced discrepancy between the training and rollout paths,

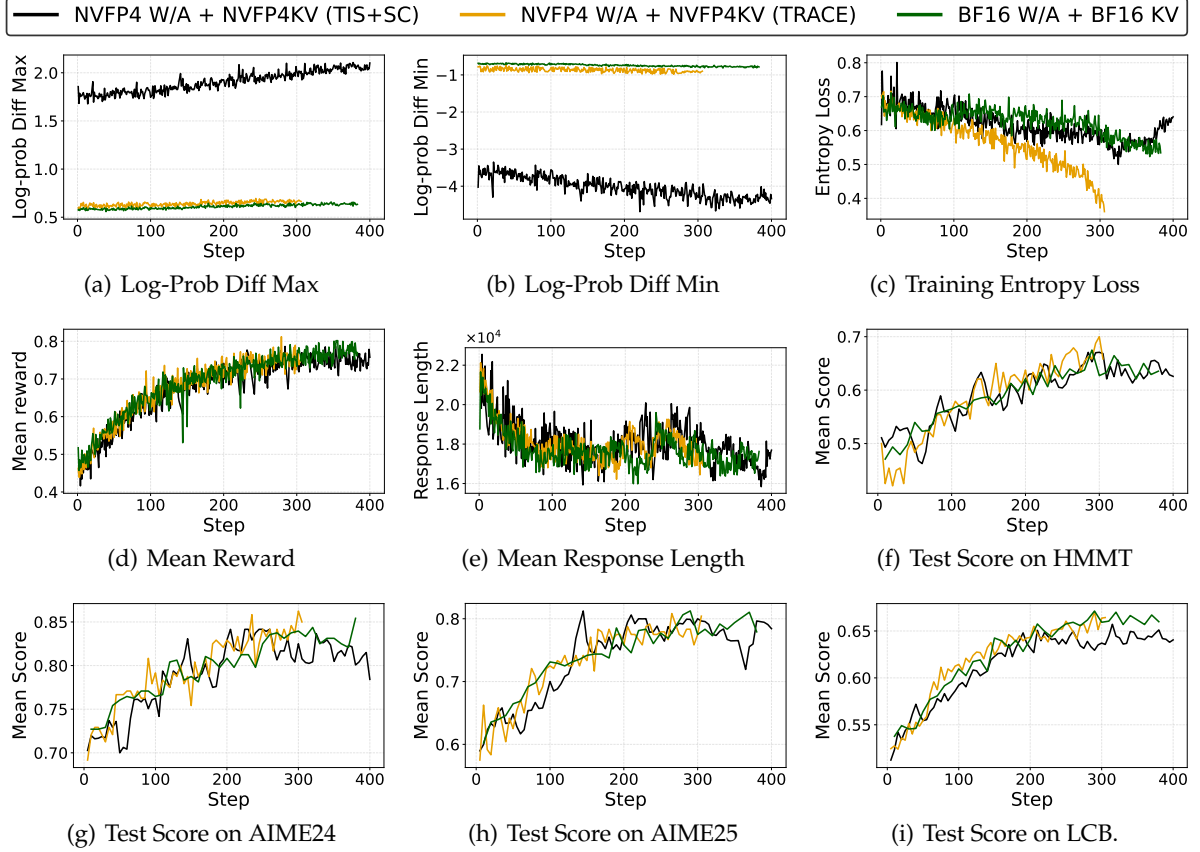


Figure 11: Training dynamics of Qwen3.5-35B-A3B on reasoning RL tasks, including the minimum/maximum log-probability difference, reward, entropy loss, response length, and test scores on HMMT, AIME24, AIME25, and LiveCodeBench.

TRACE provides both more consistent policy alignment and stronger average task performance.

4.5 Comparison with Post-Hoc FP4 Quantization

We compare the final FP4 policy performance obtained under two different training and quantization regimes. For the PTQ baselines, RL training is completed with BF16 rollout, after which different PTQ algorithms are applied post hoc to the resulting policy. In contrast, TRACE performs RL training directly with joint FP4 weight/activation and FP4 KV-cache rollout, allowing the policy to adapt to low-precision execution throughout training. As shown in Table 7, the post-hoc PTQ pipelines achieve average scores of 70.4, 71.0, and 71.4 with vanilla NVFP4, 4over6, and H-Scale, respectively, whereas TRACE achieves 75.3 under FP4 rollout throughout RL training. This result demonstrates that incorporating low-precision execution into RL training with TRACE can produce a strong final FP4 policy without relying on BF16 rollout followed by post-training quantization.

Table 7: Final FP4 policy performance ($\uparrow$) under different RL training and quantization regimes on Qwen3.5-35B-A3B. For PTQ baselines, RL training is completed with BF16 rollout before applying post-hoc FP4 quantization. In contrast, TRACE uses joint FP4 weight/activation and FP4 KV-cache rollout throughout RL training.

METHOD	LIVECODEBENCH	AIME24	AIME25	HMMT25	AVERAGE
BF16	67.1	83.8	81.3	67.5	74.9
Vanilla NVFP4	63.9	75.7	78.4	63.4	70.4
4over6	64.1	83.8	76.7	59.2	71.0
H-Scale	65.5	82.0	77.8	60.1	71.4
TRACE	66.4 ($\uparrow 0.9$)	86.3 ($\uparrow 2.5$)	78.5 ($\uparrow 0.1$)	70.0 ($\uparrow 6.6$)	75.3 ($\uparrow 3.9$)

5 Conclusion

In this paper, we presented TRACE, an FP4 quantization framework for RL training of MoE language models. TRACE combines rollout-guided QAT with efficient quantization-information caching to reduce train-rollout discrepancy and its associated overhead. Experiments across multiple MoE models and RL tasks show that TRACE enables joint FP4 weight/activation and KV-cache rollout while preserving performance comparable to BF16 rollout and substantially improving rollout efficiency.

Acknowledgements

This work was supported by Alibaba Research Intern Program.

References

- Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. MathArena: Evaluating LLMs on uncontaminated math competitions. In *NeurIPS Datasets and Benchmarks Track*, 2025.
- Jack Cook, Junxian Guo, Guangxuan Xiao, Yujun Lin, and Song Han. Four Over Six: More accurate NVFP4 quantization with adaptive block scaling. *CoRR*, abs/2512.02010, 2025.
- Daize Dong, Junlin Chen, Haolong Jia, Jiang Liu, Jiawei Wu, Huanwei Di, Jialian Wu, Zhengzhong Liu, Zicheng Liu, Emad Barsoum, Dimitris N. Metaxas, and Hongyi Wang. PR2: predictive routing replay for MoE-based LLM reinforcement learning. *CoRR*, abs/2606.00395, 2026.
- Hao Gu, Hao Wang, Jiacheng Liu, Lujun Li, Qiyuan Zhu, Bei Liu, Binxing Xu, Lei Wang, Xintong Yang, Sida Lin, Sirui Han, and Yike Guo. Qarl: Rollout-aligned quantization-aware RL for fast and stable training under training-inference mismatch. In *ACL (Findings)*, pages 17051–17064. Association for Computational Linguistics, 2026.
- Wei Huang, Yi Ge, Shuai Yang, Yicheng Xiao, Huizi Mao, Yujun Lin, Hanrong Ye, Sifei Liu, Ka Chun Cheung, Hongxu Yin, Yao Lu, Xiaojuan Qi, Song Han, and Yukang Chen. QeRL: Beyond efficiency - quantization-enhanced reinforcement learning for LLMs. *CoRR*, abs/2510.11696, 2025.
- Wenqi Huang, Charley Lee, Leonard Tng, and Serena Ge. DeepSWE: Measuring frontier coding agents on original, long-horizon engineering tasks. *CoRR*, abs/2607.07946, 2026.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *ICLR*. OpenReview.net, 2025.
- Yuhang Li, Reena Elangovan, Xin Dong, Priyadarshini Panda, and Brucek Khailany. QuRL: Efficient reinforcement learning with quantized rollout. *CoRR*, abs/2602.13953, 2026.
- Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers. *CoRR*, abs/2510.11370, 2025.
- Hei Yi Mak, Shadan Golestan, Hoang Le, Mehran Taghian Jazi, Yunke Peng, Yaoyuan Wang, Yao Wang, Junsong Wang, Tianchi Hu, Fengchen He, Guipeng Hu, Tanzila Rahman, and Anandharaju Durai Raju. HiFloat4 format for end-to-end reinforcement learning post-training of large language models. *CoRR*, abs/2607.26515, 2026.
- Martin Marek and Max Ryabinin. Score centering stabilizes off-policy reinforcement learning. *arXiv preprint arXiv:2609.20807*, 2026.
- Mathematical Association of America. American invitational mathematics examination (AIME) 2024. <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>, 2024.
- Mathematical Association of America. American invitational mathematics examination (AIME) 2025. <https://maa.org/math-competitions/american-invitational-mathematics-examination-aime>, 2025.
- Mike A. Merrill, Alexander Glenn Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, Estefany Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis-Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Kumar Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu,

-
- Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *CoRR*, abs/2601.11868, 2026.
- Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubei, Phoebe Thacker, Laurance Fauconnet, Natalie S. Kim, Patrick Chao, Samuel Miserendino, Gildas Chabot, David Li, Michael Sharman, Alexandra Barr, Amelia Glaese, and Jerry Tworek. GDPval: Evaluating AI model performance on real-world economically valuable tasks. *CoRR*, abs/2510.04374, 2025.
- Zihan Qiu, Zekun Wang, Xiao Li, Yanpeng Li, Yang Xu, Yixuan Wang, Huaqing Zhang, Rui Men, Bochao Mao, Chengruidong Zhang, Fan Zhou, Hao Luo, Haofeng Huang, Haoran Lian, Haoyan Huang, Hongqing Chen, Jianwei Zhang, Jing Xu, Junjie Wang, Langshi Chen, Liangyu Wang, Linlang Jiang, Man Yuan, Minmin Sun, Peng Jin, Siqi Zhang, Siyu Wang, Xingzhang Ren, Yakai Wang, Yi Zhang, Yiming Dong, Yizhong Cao, Yubo Ma, Yunfei Mao, Bo Zheng, and Dayiheng Liu. On the design of Qwen3.8-Next architecture: Evaluation, efficiency, and training stability. *CoRR*, abs/2608.30320, 2026.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026a. URL <https://qwen.ai/blog?id=qwen3.5>.
- Qwen Team. Qwen3.8-Max: A new bar for coding and cowork, August 2026b. URL <https://qwen.ai/blog?id=qwen3.8>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarczyk, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR (Poster)*. OpenReview.net, 2017.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. In *EuroSys*, pages 1279–1297. ACM, 2025.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *CoRR*, abs/1909.08053, 2019.
- Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Zihe Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *CoRR*, abs/2506.06122, 2025.
- Xin Wang, Zhongwei Wan, Arvin Hekmati, Mingyu Zong, Samiul Alam, Mi Zhang, and Bhaskar Krishnamachari. IoT in the era of generative AI: vision and challenges. *CoRR*, abs/2401.01923, 2024.
- Haocheng Xi, Charlie Ruan, Peiyuan Liao, Yujun Lin, Han Cai, Yilong Zhao, Shuo Yang, Kurt Keutzer, Song Han, and Ligeng Zhu. Jet-RL: Enabling on-policy FP8 reinforcement learning with unified training and rollout precision flow. *CoRR*, abs/2601.14243, 2026.
- Jinghan Yao, Eric Tang, Sumanth Hegde, and Kourosh Hakhmaneshi. FP8 reinforcement learning in SkyRL: Preserving policy consistency across training and rollout. <https://www.anyscale.com/blog/fp8-reinforcement-learning-in-skyrl>, 2026. Accessed: 2026-09-02.
- Hao Yu, Zheng Li, Dayiheng Liu, and Jianwei Zhang. H-Scale: hessian-guided scale refinement for NVFP4 sub-byte LLM inference. *CoRR*, abs/2608.28113, 2026.
- Kaiyan Zhang, Yuxin Zuo, Bingxiang He, Youbang Sun, Runze Liu, Che Jiang, Yuchen Fan, Kai Tian, Guoli Jia, Pengfei Li, Yu Fu, Xingtai Lv, Yuchen Zhang, Sihang Zeng, Shang Qu, Haozhan Li, Shijie Wang, Yuru Wang, Xinwei Long, Fangfu Liu, Xiang Xu, Jiaze Ma, Xuekai Zhu, Ermo Hua, Yihao Liu, Zonglin Li, Huayu Chen, Xiaoye Qu, Yafu Li, Weize Chen, Zhenzhao Yuan, Junqi Gao, Dong Li, Zhiyuan Ma, Ganqu Cui, Zhiyuan Liu, Biqing Qi, Ning Ding, and Bowen Zhou. A survey of reinforcement learning for large reasoning models. *CoRR*, abs/2509.08827, 2025.

-
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models. *CoRR*, abs/2303.18223, 2023.
- Chujie Zheng, Kai Dang, Bowen Yu, Mingze Li, Huiqiang Jiang, Junrong Lin, Yuqiong Liu, Hao Lin, Chencan Wu, Feng Hu, An Yang, Jingren Zhou, and Junyang Lin. Stabilizing reinforcement learning with LLMs: Formulation and practices. *CoRR*, abs/2512.01374, 2025a.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *CoRR*, abs/2507.18071, 2025b.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark W. Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *NeurIPS*, 2024.
- Zhengyang Zhuge, Hao Yu, Xin Wang, Zheng Li, Yizhong Cao, Dayiheng Liu, and Jianwei Zhang. QUADS: stabilizing NVFP4 reinforcement learning for MoE via QUantization-error alignment across dual sides. *CoRR*, abs/2607.15810, 2026.

A Appendix.

A.1 Pseudocode of TRACE

Algorithm 1 summarizes TRACE for quantized MoE inputs and KV states. The mantissa-only setting uses $b = 1$ in the latter half of the layers. Each record contains packed low-bit rollout codes and the corresponding rollout scales, indexed by trajectory, token, and site. During training, these records are used to construct a compact rollout-side reference for guiding the local FP4 rounding decision.

Algorithm 1 TRACE: Rollout-Guided FP4 QAT

Require: Policy parameters W , prompts $\mathcal{D}$, reward function r , captured sites $\mathcal{S}$, bits $b = 1$

Ensure: Updated policy parameters W

```
1: for each RL iteration do
2:   Generate trajectories  $\mathcal{T}$  and log-probabilities  $\ell_{\text{roll}}$  from  $\mathcal{D}$  using  $Q_{\text{FP4}}(W)$ 
3:   During rollout, record  $\mathcal{R} = \{(\text{Pack}(\text{low}_b(q_{\text{roll}})), s_{\text{roll}})\}$  at  $\mathcal{S}$ 
4:   Compute advantages  $A$  from rewards  $r(\mathcal{T})$ 
5:   for each policy-update minibatch  $\mathcal{B} \subseteq \mathcal{T}$  do
6:     for each captured activation  $x$  encountered during the QAT forward do
7:        $(c, s) \leftarrow \text{UNPACK}(\mathcal{R}[\text{trajectory}, \text{token}, \text{site}])$ 
8:        $z \leftarrow \text{clip}(x/s, -6, 6)$   $\triangleright$  Normalize with the rollout scale
9:        $\hat{q}_{\text{roll}} \leftarrow \text{RECONSTRUCT}(c, b, z)$ 
10:       $(q_-, q_+) \leftarrow \text{BRACKETFP4}(z)$ 
11:       $q^* \leftarrow \arg \min_{q \in \{q_-, q_+\}} |q - \hat{q}_{\text{roll}}|$ 
12:      Use  $\text{STE}(x, sq^*)$  in the remaining forward computation
13:    end for
14:    Compute  $\mathcal{L}_{\text{RL}}$  from the QAT log-probabilities,  $\ell_{\text{roll}}$ , and  $A$  on  $\mathcal{B}$ 
15:     $W \leftarrow \text{OPTIMIZERSTEP}(W, \nabla_W \mathcal{L}_{\text{RL}})$ 
16:  end for
17: end for
18: return  $W$ 
```

Here s combines block and token/global scales. RECONSTRUCT constructs a compact rollout-side reference by selecting the E2M1 magnitude nearest to $|z|$ whose low b code bits match c , while using the sign of the current training activation. Accordingly, the reconstructed value is used as a local guidance target rather than as an exact recovery of the original rollout-side FP4 codeword. BRACKETFP4 returns the adjacent E2M1 rounding candidates around z , and STE returns its second argument in the forward pass while passing gradients to its first argument.

A.2 Training Dynamics on Qwen3.8-Flash-Next

Figure 12 complements the main-text analysis of coding RL on Qwen3.8-Flash-Next. We compare vanilla QAT and TRACE under joint NVFP4 weight and KV-cache quantization with a BF16 rollout reference. The six panels show the same training diagnostics as in the main text, together with test scores on Terminal-Bench 2.1 (TB), over a window of 400 training steps.

Training–Rollout Consistency. In the recorded training steps, vanilla QAT exhibits larger log-probability differences than both TRACE and the BF16 reference. Its maximum difference increases and its minimum becomes more negative toward the end of the recorded run. In contrast, TRACE remains much closer to the BF16 reference in both diagnostics, with no comparable widening over the 400-step window. This behavior is consistent with the improved training–rollout alignment observed on reasoning RL tasks.

Reward, Entropy, and Response Length. Vanilla QAT nevertheless exhibits a pronounced entropy decrease after approximately step 100 and shorter responses near the end of its recorded trajectory. TRACE shows a more gradual entropy decrease, maintains long responses, and improves reward on average. Its recorded TB test scores also increase over the training window. These observations support the training-stability trend in the main text.

A.3 Training Dynamics on Qwen3.8-2.4T-A95B

Figure 13 complements the main-text comparison of long-horizon RL on Qwen3.8-2.4T-A95B. We compare vanilla QAT and TRACE under joint NVFP4 weight and KV-cache quantization with a BF16 rollout reference. The four panels report the maximum log-probability difference, mean reward, training entropy loss, and mean response length over the 50-step training run.

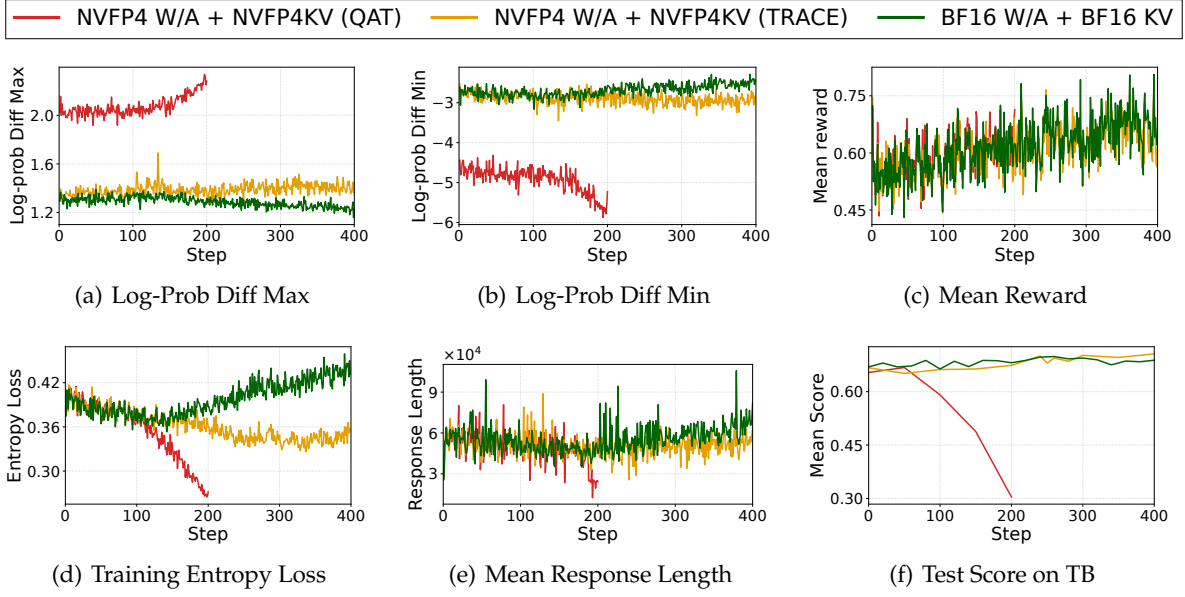


Figure 12: Performance evolution of Qwen3.8-Flash-Next on coding RL tasks: maximum and minimum log-probability differences, mean reward, training entropy loss, mean response length, and test score on Terminal-Bench 2.1.

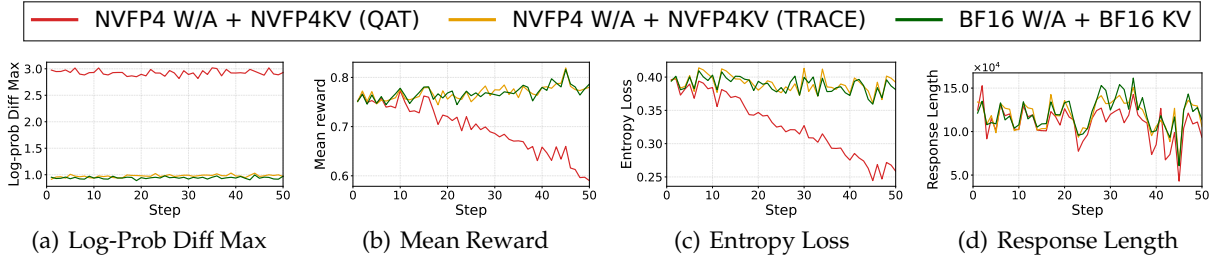


Figure 13: Training dynamics for Qwen3.8-2.4T-A95B (Max): (a) maximum log-probability difference, (b) mean reward, (c) training entropy loss, and (d) mean response length.

Training-Rollout Consistency. Over the 50 recorded steps, vanilla QAT keeps a larger maximum log-probability difference, near 2.9, while TRACE and the BF16 reference stay near 0.95. TRACE remains close to the BF16 reference throughout the window. This behavior is consistent with the improved training-rollout alignment observed on reasoning and coding RL tasks.

Reward, Entropy, and Response Length. For Vanilla QAT, the mean reward falls from about 0.75 to about 0.59, training entropy falls from about 0.40 to about 0.26, and mean response length shortens from about 125K to about 93K tokens. TRACE raises mean reward from about 0.75 to about 0.78, keeps entropy near the BF16 reference at about 0.38–0.40, and keeps responses in the 106K–134K range. These observations support the training-stability trend reported for long-horizon RL in the main paper.

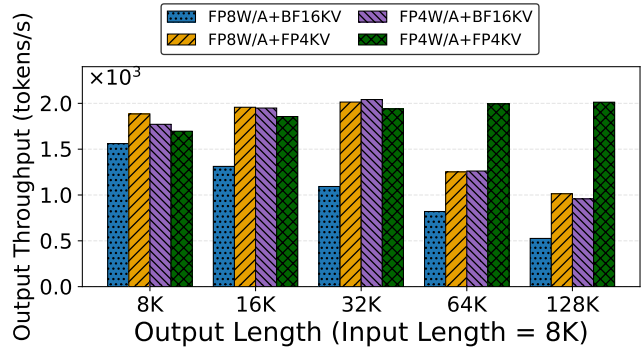


Figure 14: Qwen3.8-Flash-Next output throughput under four expert-weight/KV precision combinations. All settings use one GB200, BF16 PLE offload, 8K-token inputs, and 64 submitted requests.

A.4 Throughput Analysis on Qwen3.8-Flash-Next

Setup and Metric. Following Figure 9(a), we compare four routed-expert/KV precision combinations on Qwen3.8-Flash-Next: FP8/BF16, FP8/FP4, FP4/BF16, and FP4/FP4. All use the same checkpoint on one GB200, BF16 PLE/ngram embeddings offloaded to CPU pinned memory, and no multi-token prediction.

The GPU memory fraction is 0.85, with automatic KV allocation. Each measurement submits 64 synthetic 8K-token prompts and generates exactly 8K–128K output tokens per request.

Dependence on Output Length. In Figure 14, joint FP4 improves throughput over FP8/BF16 by $1.09\times$, $1.41\times$, $1.77\times$, and $2.43\times$ at 8K, 16K, 32K, and 64K outputs, respectively. In the single-run 128K supplement, joint FP4 reaches 2013 tokens/s versus 526 tokens/s for FP8/BF16 ($3.83\times$). The corresponding peak decoding batches are 64 and 7 requests, so this gain includes the concurrency enabled by KV capacity.